

Name of Student : S.Suhas

Enrollment.No:2303A51453

Batch:21

Task 1 – Fairness in AI Code Generation

- **Provide AI with tasks involving different user groups (e.g., age, gender).**
- **Compare outputs for fairness.**
- **Discuss potential bias and its impact.**

Code:-

```
# Task-01
# Generate a Python function that recommends personalized workout plans. First, create one for a 25-
# Function to recommend workout plan for a 25-year-old male office worker
def recommend_workout_plan_male(age, weight, height):
    # Placeholder implementation - replace with actual workout recommendation logic
    return f"Workout plan for {age}-year-old male office worker"

# Function to recommend workout plan for a 25-year-old female office worker
def recommend_workout_plan_female(age, weight, height):
    # Placeholder implementation - replace with actual workout recommendation logic
    return f"Workout plan for {age}-year-old female office worker"

# Example usage
male_workout_plan = recommend_workout_plan_male(25, 70, 175)
female_workout_plan = recommend_workout_plan_female(25, 60, 165)
print(f"Male workout plan: \n{male_workout_plan}")
print(f"Female workout plan: \n{female_workout_plan}")

# Reflection on Gender Bias
# When comparing the two workout plans, it's important to analyze the exercise selection, intensity,
# assumptions made in the recommendations. If the male workout plan includes more strength training
```

Output:-

```
Male workout plan:
Workout plan for 25-year-old male office worker
Female workout plan:
Workout plan for 25-year-old female office worker
PS C:\AIAC LAB\Assignments Codes>
```

Task 2 – Transparency of AI Decisions

- Use AI to generate a recommendation system.
- Analyze whether the logic is explainable.
- Discuss the need for transparency in AI-generated code.

Code:-

```
# Task - 02
# Build a simple movie recommendation system in Python using a dataset of 5 sample users and movies with ratings. Use a
# Sample dataset of users, movies, and ratings
users = {
    "User1": {"MovieA": 5, "MovieB": 3, "MovieC": 4},
    "User2": {"MovieA": 4, "MovieB": 2, "MovieD": 5},
    "User3": {"MovieB": 5, "MovieC": 3, "MovieD": 4},
    "User4": {"MovieA": 2, "MovieC": 5, "MovieD": 3},
    "User5": {"MovieB": 4, "MovieC": 2, "MovieD": 5},
}

# Function to calculate similarity between two users based on their ratings
def calculate_similarity(user1, user2):
    common_movies = set(users[user1].keys()) & set(users[user2].keys())
    if not common_movies:
        return 0
    ratings1 = [users[user1][movie] for movie in common_movies]
    ratings2 = [users[user2][movie] for movie in common_movies]
    similarity = sum((r1 - r2) ** 2 for r1, r2 in zip(ratings1, ratings2)) ** 0.5
    return similarity

# Function to recommend movies for a given user
def recommend_movies(user):
    similarities = {other_user: calculate_similarity(user, other_user) for other_user in users if other_user != user}
    most_similar_user = max(similarities, key=similarities.get)
    recommended_movies = [movie for movie in users[most_similar_user] if movie not in users[user]]
    return recommended_movies

# Example usage
user_to_recommend = "User1"
recommended_movies = recommend_movies(user_to_recommend)
print(f"Recommended movies for \n{user_to_recommend}: {recommended_movies}")

# Explanation of Recommendation Logic
# The recommendation logic works by first calculating the similarity between the target user and all other users based
```

Output:-

```
Recommended movies for
User1: ['MovieD']
```

Task 3 – Data Misuse and Ethical Risks

- Ask AI to process datasets containing user information.
- Identify risks of misuse or leakage.
- Propose safeguards.

Prompt:-

Code:-

```
# Task - 03
# Here's a sample dataset of user information: User1 (name: Alice, age: 30, email: alice@email.com)
# Sample dataset of user information
users = [
    {"name": "Alice", "age": 30, "email": "alice@email.com", "salary": 50000},
    {"name": "Bob", "age": 35, "email": "bob@email.com", "salary": 60000}
]

# Analyze average salary by age group
age_groups = {}
for user in users:
    age_group = user["age"] // 10 * 10 # Group by tens
    if age_group not in age_groups:
        age_groups[age_group] = []
    age_groups[age_group].append(user["salary"])

# Calculate and output average salary for each age group
for age_group, salaries in age_groups.items():
    average_salary = sum(salaries) / len(salaries)
    print(f"Average salary for age group {age_group}-{age_group + 9}: \n${average_salary:,.2f}")

# Potential risks of data leakage or misuse:
# 1. Direct identification of individuals through email addresses.
# 2. Sensitive information (salary) is exposed in the analysis.
# Safeguards:
# 1. Anonymize user data by removing or hashing personal identifiers.
# 2. Implement access controls to limit who can view the analysis results.
# 3. Use encryption to protect data at rest and in transit.
```

Output:-

```
Average salary for age group 30-39:
$55,000.00
PS C:\AIAC LAB\Assignments Codes>
```

Task 4 – AI Hallucination in Coding

- Ask AI to generate complex algorithms.
- Verify correctness of output.
- Identify incorrect or misleading code.

Code:-

```
# Task - 04
# Implement Kruskal's algorithm for Minimum Spanning Tree

class DisjointSet:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, u):
        if self.parent[u] != u:
            self.parent[u] = self.find(self.parent[u])
        return self.parent[u]

    def union(self, u, v):
        root_u = self.find(u)
        root_v = self.find(v)
        if root_u != root_v:
            if self.rank[root_u] > self.rank[root_v]:
                self.parent[root_v] = root_u
            elif self.rank[root_u] < self.rank[root_v]:
                self.parent[root_u] = root_v
            else:
                self.parent[root_v] = root_u
                self.rank[root_u] += 1

def kruskal(num_nodes, edges):
    disjoint_set = DisjointSet(num_nodes)
    mst_edges = []
    total_weight = 0

    # Sort edges based on weight
    edges.sort(key=lambda x: x[2])
```

```

    for u, v, weight in edges:
        if disjoint_set.find(u) != disjoint_set.find(v):
            disjoint_set.union(u, v)
            mst_edges.append((u, v, weight))
            total_weight += weight

    return mst_edges, total_weight
# Graph edges (u, v, weight)
edges = [
    (0, 1, 2),
    (0, 3, 6),
    (1, 2, 3),
    (1, 3, 5),
    (2, 3, 4),
    (2, 4, 5),
    (3, 4, 3)
]
num_nodes = 5
mst_edges, total_weight = kruskal(num_nodes, edges)
print("MST Edges:")
for u, v, weight in mst_edges:
    print(f"({u}, {v}): {weight}")
print(f"Total Weight: \n{total_weight}")
# Manual tracing of Kruskal's algorithm:
# 1. Sort edges by weight: (0,1:2), (1,2:3), (3,4:3), (2,3:4), (1,3:5), (2,4:5), (0,3:6)
# 2. Start with the smallest edge (0,1:2) - add to MST
# 3. Next edge (1,2:3) - add to MST
# 4. Next edge (3,4:3) - add to MST
# 5. Next edge (2,3:4) - add to MST
# 6. Next edge (1,3:5) - skip (creates a cycle)
# 7. Next edge (2,4:5) - skip (creates a cycle)
# 8. Next edge (0,3:6) - skip (creates a cycle)
# Final MST edges: (0,1:2), (1,2:3), (3,4:3), (2,3:4)
# Total weight: 2 + 3 + 3 + 4 = 12

```

Output:-

```

MST Edges:
(0, 1): 2
(1, 2): 3
(3, 4): 3
(2, 3): 4
Total Weight:
12

```

Task 5– Responsibility in AI Deployment

- Scenario: AI code is deployed in a banking system.
- Identify risks and accountability.
- Propose ethical guidelines.

Code:-

```
# Task - 05
# Imagine deploying an AI-generated fraud detection model in a banking system. The model uses logistic regression
import numpy as np
from pyarsing import nums
from sklearn.linear_model import LogisticRegression
# Sample dataset of transactions (amount, location, time) and labels (0 for legitimate, 1 for fraudulent)
X = np.array([[100, 1, 12], [200, 2, 14], [150, 1, 16], [300, 3, 18], [250, 2, 20]])
y = np.array([0, 1, 0, 1, 0])

# Create and train the logistic regression model
model = LogisticRegression()
model.fit(X, y)
# Function to predict fraud for a new transaction
def predict_fraud(transaction):
    prediction = model.predict([transaction])
    return "Fraudulent" if prediction[0] == 1 else "Legitimate"
# Example usage
new_transaction = [180, 2, 15] # amount, location, time
print(f"Prediction for new transaction: \n{predict_fraud(new_transaction)}")
# Risks and Accountability:
# 1. False positives can lead to innocent users being flagged as fraudulent, which can cause inconvenience and damage to the bank's reputation.
# 2. Accountability can be complex, as it may involve the developer who created the model, the AI provider that supplied the model, and the bank that deployed it.
# Ethical Guidelines:
# - Regular audits of the model's performance and decision-making process.
# - Human oversight in critical decisions, especially when the model flags transactions as potentially fraudulent.
# - Transparent communication about how the model works and the basis for its predictions.
```

Output:-

```
Prediction for new transaction:
Legitimate
```

Task 6 – Ethical Issues in Automation

- Use AI to automate a decision-making system (e.g., hiring filter).
- Analyze fairness and ethical concerns.
- Suggest improvements.

Code:-

```
# Task - 06
# Create a Python hiring filter system that scores resumes based on keywords
# Sample resumes
resumes = [
    {"name": "Candidate1", "skills": ["Python", "Java"], "experience": 2},
    {"name": "Candidate2", "skills": ["management"], "experience": 5}
]

# Function to score resumes based on keywords and experience
def score_resume(resume):
    score = 0
    keywords = ['Python', 'experience', 'degree']
    for keyword in keywords:
        if keyword in resume['skills']:
            score += 1
    score += resume['experience'] # Add years of experience to the score
    return score

# Rank candidates based on their scores
ranked_candidates = sorted(resumes, key=score_resume, reverse=True)
print("Ranked Candidates:")
for candidate in ranked_candidates:
    print(candidate['name'])

# Fairness Issues and Improvements:
# 1. The scoring system may be biased against non-technical roles or candidates
# 2. To improve fairness, the training data for the scoring system should be
```

Output:-

```
Ranked Candidates:
Candidate2
Candidate1
```

Task 7 – AI and Intellectual Property

- Generate code using AI for a known problem.
- Check similarity with existing solutions.
- Discuss plagiarism and copyright concerns.

Code:-

```
# Task - 07
# Generate Python code to solve the LeetCode 'Two Sum' problem: Given num
def two_sum(nums, target):
    num_to_index = {}
    for index, num in enumerate(nums):
        complement = target - num
        if complement in num_to_index:
            return [num_to_index[complement], index]
        num_to_index[num] = index
    return []

nums = [2, 7, 11, 15]
target = 9
result = two_sum(nums, target)
print(f"Indices of the two numbers that add up to {target}: \n{result}")
# Comparison to Standard Solution:
# The generated code structure and logic closely resemble the standard so
# Plagiarism Risks and IP Concerns:
# While the code structure is similar to the standard solution, it includ
```

Output:-

```
Indices of the two numbers that add up to 9:
[0, 1]
```